ENGINEERING BRIEF · CONFIDENTIAL

Wellness Booking Platform Technical Framework

Architecture, data model, scheduling engine and integration specification for the Mombasa studio & mobile massage service

Audience: Engineering team

Stack: Next.js · Supabase (Postgres) · M-Pesa Daraja · WhatsApp Cloud API

Version: 1.0 — September 2026

Status: For review before sprint planning

Contents

1	What we are building, and the one hard part
2	System architecture
3	Hosting decision: Supabase, Vercel, Railway — and why not cPanel
4	Data model and schema
5	Row Level Security model
6	The availability & scheduling engine
7	Booking transaction and double-booking prevention
8	Payments: M-Pesa STK Push, cards, payouts
9	WhatsApp, notifications and number masking
10	Edge Functions and background jobs
11	Security, privacy and Kenyan data-protection obligations
12	Environments, CI/CD and testing
13	12-week delivery plan
14	Definition of done

1. What we are building, and the one hard part

> A booking, payment and dispatch platform for a premium massage service operating from a studio in Mombasa and delivering to client homes and hotel rooms. Three user types, one database, one deployment.

1.1 The three surfaces

Surface	Users	What it does
Client web app Public, PWA	Residents & tourists	Browse treatments, check availability, pick therapist, book, pay, receive confirmations, rate, rebook
Therapist app Auth, mobile web	4–10 therapists	Set availability, accept jobs, navigate, check in/out with GPS, panic button, view earnings
Ops console Auth, desktop web	1–2 ops staff	Live board, manual assignment, exception handling, incident log, payout runs, client and therapist records

1.2 The hard part

Ninety percent of this system is ordinary CRUD. One part is not: **computing which therapists are genuinely available for a given treatment, time and location**, given that they move between the studio, client homes across Mombasa, and hotels along the North Coast.

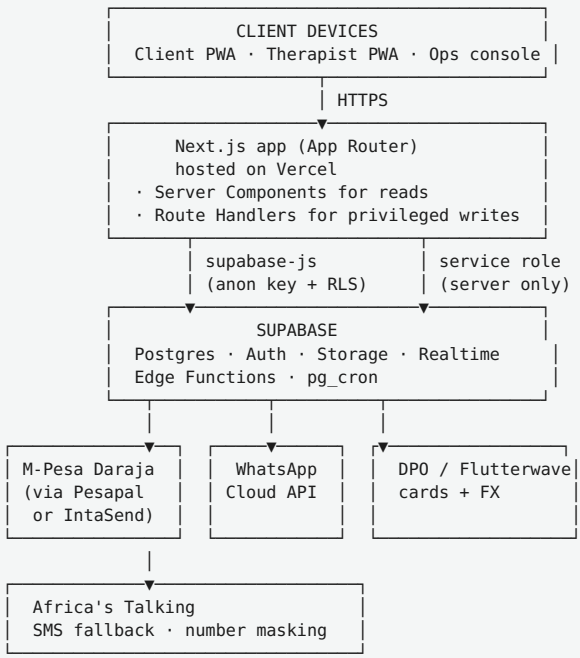
A therapist finishing a session in Nyali at 14:00 cannot begin one in Shanzu at 14:15. A therapist being free does not help if both studio rooms are occupied. Not every therapist is trained in prenatal or deep tissue. The client may have requested a specific therapist gender.

> **WHERE PROJECTS LIKE THIS FAIL**
>
> Double bookings and impossible travel schedules. Both destroy trust with the client *and* the therapist, and both are avoidable at the database layer rather than in application code. Section 6 and 7 specify how. Please read those two sections before writing any booking logic.

1.3 Non-goals for version one

Explicitly out of scope, do not build: native iOS/Android apps, therapist self-onboarding, memberships and gift vouchers, a corporate portal, loyalty points, multi-language, automated payout execution, review syndication to Google, or any form of surge/dynamic pricing. All are phase two. Scope creep here will cost us the launch window.

2. System architecture



2.1 Component responsibilities

Component	Responsibility
Next.js / Vercel	All three UIs. Server Components read through the user's own session so RLS applies. Route Handlers hold anything needing the service role or a third-party secret.
Supabase Postgres	Single source of truth. Business invariants enforced as constraints, not application code.
Supabase Auth	Phone OTP for clients and therapists, email+password for ops. One `profiles` row per `auth.users` row carrying the role.
Supabase Edge Functions	Public webhook receivers (payment callbacks, WhatsApp inbound). These must be reachable by third parties, so they cannot live behind our auth.
pg_cron	Reminder dispatch, payment reconciliation sweep, availability materialisation. Replaces a separate worker service at our volume.
Supabase Realtime	Ops console live board only. Do not use it for the client booking flow.
Supabase Storage	Therapist photos, credential scans, treatment imagery. Credential scans go in a private bucket.

> **A RULE TO HOLD TO**
>
> The service role key never leaves the server. Not in a Client Component, not in `NEXT_PUBLIC_`, not in the therapist PWA. If a feature seems to need it client-side, the feature is designed wrong — move it into a Route Handler or an RPC.

3. Hosting decision

The brief mentioned cPanel or Railway. Here is a direct recommendation, because this choice is expensive to reverse.

Recommended

Layer	Choice	Reasoning
Database, auth, storage, functions	**Supabase**	Already chosen, and correct. Managed Postgres with RLS, auth and edge functions in one place removes most of our infrastructure work.
Web application	**Vercel**	Next.js is a Vercel product. Zero-config deploys, preview URLs per pull request, edge caching, automatic HTTPS. Free tier covers us until real traffic.
Background workers (only if needed)	**Railway**	Good fit for long-running jobs if we outgrow pg_cron and Edge Functions — for example a payout batch runner. Not needed at launch.
Marketing site / domain email	cPanel (if we already pay for it)	Fine for a static brochure page and mailboxes. Point the app at a subdomain.

> **DO NOT HOST THE APPLICATION ON CPANEL**
>
> cPanel is built for PHP on shared Apache hosting. Running Next.js on it means Passenger or a hand-managed Node process, and in practice that gives us: no zero-downtime deploys, no preview environments, fragile process restarts after any panel update, awkward environment-variable handling, cron limited to minute granularity with no observability, and no straightforward rollback. Every hour saved by reusing existing hosting will be repaid several times over in deployment incidents. If there is an existing cPanel contract, use it for the marketing page and email only.

3.1 Supabase project setup

- Region: **eu-central-1 (Frankfurt)** or **ap-south-1 (Mumbai)**. Mumbai typically gives lower latency to Kenya; test both with a ping before committing, since we cannot change region later without a migration.
- Two projects: `wellness-staging` and `wellness-prod`. Never develop against production.
- Enable Point-in-Time Recovery on production before launch day. This is a paid add-on and it is not optional — we hold booking and payment records.
- Enable `pg_cron`, `pgcrypto`, `btree_gist` and `postgis` (postgis only if we later need real geo queries; the zone table in §6 avoids it for now).

4. Data model and schema

4.1 Entity overview

```
profiles ──┬── clients ──────< bookings >────── therapists ──┬── therapist_credentials
           │       │              │                          ├── therapist_skills
           │       │              ├── payments                ├── availability_windows
           │       │              ├── session_logs            └── payouts
           │       │              ├── ratings
           │       │              └── incidents
           │       └── client_verifications
           └── ops staff

treatments ──< treatment_pricing (by delivery mode)
rooms (studio treatment rooms)
zones ──< zone_travel_matrix
partners (hotels, corporates) ──< bookings
```

4.2 Core enums and reference tables

```
create type user_role      as enum ('client','therapist','ops','admin');
create type delivery_mode  as enum ('studio','home','hotel');
create type booking_status as enum ('pending_payment','confirmed','in_progress',
                                    'completed','cancelled','no_show');
create type booking_origin as enum ('app','hotel_partner','walk_in','phone','corporate');
create type payment_status as enum ('pending','succeeded','failed','refunded','timed_out');

-- Travel zones. Deliberately coarse; see section 6.3.
create table zones (
  id    text primary key,          -- 'island','nyali','bamburi','shanzu','mtwapa'
  name  text not null,
  is_serviceable boolean not null default true
);

create table zone_travel_matrix (
  from_zone text references zones(id),
  to_zone   text references zones(id),
  minutes   int not null check (minutes >= 0),
  primary key (from_zone, to_zone)
);
```

4.3 People

```
create table profiles (
  id          uuid primary key references auth.users(id) on delete cascade,
  role        user_role not null default 'client',
  full_name   text not null,
  phone       text unique not null,
  created_at  timestamptz not null default now()
);

create table clients (
  id                uuid primary key references profiles(id) on delete cascade,
  email             text,
  -- Commercial evidence. See the partnership agreement: we must be able to
  -- prove which clients the platform introduced.
  acquired_via      text not null default 'app',
  is_pre_existing   boolean not null default false,
  id_document_type  text,      -- 'national_id' | 'passport'
  id_document_no    text,      -- encrypted at rest, see section 11
  phone_verified_at timestamptz,
  id_verified_at    timestamptz,
  payment_verified_at timestamptz,
  is_banned         boolean not null default false,
  ban_reason        text,
  notes             text
);

create table therapists (
  id                uuid primary key references profiles(id) on delete cascade,
  display_name      text not null,
  bio               text,
  photo_path        text,             -- Supabase Storage, public bucket
  gender            text,             -- for client preference matching
  home_zone         text references zones(id),
  serves_home       boolean not null default false,
  serves_hotel      boolean not null default false,
  serves_studio     boolean not null default true,
  commission_rate   numeric(4,3) not null default 0.620,  -- therapist share
  is_active         boolean not null default false,       -- false until vetted
  vetted_at         timestamptz,
  masked_phone      text              -- Africa's Talking proxy number
);

create table therapist_credentials (
  id             uuid primary key default gen_random_uuid(),
  therapist_id   uuid not null references therapists(id) on delete cascade,
  kind           text not null,   -- 'tvet_cdacc','knec','good_conduct','insurance','health'
  issuer         text,
  reference_no   text,
  issued_on      date,
  expires_on     date,
  scan_path      text,            -- PRIVATE storage bucket
  verified_by    uuid references profiles(id),
  verified_at    timestamptz
);
```

WHY CREDENTIALS ARE THEIR OWN TABLE

They expire. A therapist whose good-conduct certificate or insurance has lapsed must drop out of the availability query automatically, not by someone remembering. Section 6.2 encodes that.

4.4 Catalogue and pricing

```
create table treatments (
  id            uuid primary key default gen_random_uuid(),
  slug          text unique not null,
  name          text not null,
  description   text,
  duration_min  int not null,           -- service time only
  turnaround_min int not null default 15, -- room reset / linen change
  requires_skill text,                  -- null, or matches therapist_skills.skill
  is_active     boolean not null default true,
  sort_order    int not null default 0
);

create table treatment_pricing (
  treatment_id  uuid references treatments(id) on delete cascade,
  mode          delivery_mode not null,
  price_kes     int not null,
  is_available  boolean not null default true,
  primary key (treatment_id, mode)
);

create table therapist_skills (
  therapist_id uuid references therapists(id) on delete cascade,
  skill        text not null,  -- 'deep_tissue','prenatal','hot_stone','sports','reflexology'
  primary key (therapist_id, skill)
);

create table rooms (
  id        uuid primary key default gen_random_uuid(),
  name      text not null,        -- 'Room 1', 'Room 2'
  is_active boolean not null default true
);
```

4.5 Availability and bookings

The two range columns on `bookings` are the heart of the system. `service_range` is when the treatment actually happens. `blocked_range` extends it by turnaround and outbound travel, and is what conflict detection uses.

```
create table availability_windows (
  id           uuid primary key default gen_random_uuid(),
  therapist_id uuid not null references therapists(id) on delete cascade,
  window       tstzrange not null,
  is_blocked   boolean not null default false,  -- true = time off, false = working
  note         text,
  exclude using gist (therapist_id with =, window with &&)
);

create table bookings (
  id             uuid primary key default gen_random_uuid(),
  reference      text unique not null,            -- human-friendly, e.g. 'MB-4T7K'
  client_id      uuid not null references clients(id),
  therapist_id   uuid not null references therapists(id),
  treatment_id   uuid not null references treatments(id),
  mode           delivery_mode not null,
  status         booking_status not null default 'pending_payment',
  origin         booking_origin not null default 'app',
  partner_id     uuid references partners(id),

  service_range  tstzrange not null,
  blocked_range  tstzrange not null,              -- service + turnaround + travel

  room_id        uuid references rooms(id),       -- required when mode = 'studio'
  zone_id        text references zones(id),       -- required when mode != 'studio'
  address_line   text,
  address_notes  text,
  hotel_name     text,
  hotel_room_no  text,

  price_kes      int not null,
  therapist_share_kes int,                        -- frozen at completion
  platform_share_kes  int,

  created_at     timestamptz not null default now(),
  confirmed_at   timestamptz,
  cancelled_at   timestamptz,
  cancelled_by   uuid references profiles(id),
  cancellation_fee_kes int not null default 0,

  constraint studio_needs_room check (mode <> 'studio' or room_id is not null),
  constraint mobile_needs_zone check (mode =  'studio' or zone_id is not null),
  constraint blocked_contains_service check (blocked_range @> service_range)
);
```

4.6 Sessions, money, safety

```
create table session_logs (
  booking_id     uuid primary key references bookings(id) on delete cascade,
  checked_in_at  timestamptz,
  check_in_lat   numeric(9,6),
  check_in_lng   numeric(9,6),
  checked_out_at timestamptz,
  check_out_lat  numeric(9,6),
  check_out_lng  numeric(9,6),
  therapist_notes text
);

create table payments (
  id                  uuid primary key default gen_random_uuid(),
  booking_id          uuid not null references bookings(id),
  provider            text not null,             -- 'pesapal','intasend','dpo'
  method              text not null,             -- 'mpesa','card'
  amount_kes          int not null,
  currency            text not null default 'KES',
  status              payment_status not null default 'pending',
  provider_ref        text,                      -- CheckoutRequestID / order tracking id
  provider_receipt    text,                      -- M-Pesa receipt number
  raw_callback        jsonb,
  created_at          timestamptz not null default now(),
  settled_at          timestamptz,
  constraint uniq_provider_ref unique (provider, provider_ref)
);

create table payouts (
  id             uuid primary key default gen_random_uuid(),
  therapist_id   uuid not null references therapists(id),
  period_start   date not null,
  period_end     date not null,
  gross_kes      int not null,
  commission_kes int not null,
  tips_kes       int not null default 0,
  net_kes        int not null,
  status         text not null default 'draft',  -- draft|approved|sent|failed
  mpesa_ref      text,
  approved_by    uuid references profiles(id),
  sent_at        timestamptz,
  unique (therapist_id, period_start, period_end)
);

create table incidents (
  id            uuid primary key default gen_random_uuid(),
  booking_id    uuid references bookings(id),
  raised_by     uuid references profiles(id),
  kind          text not null,   -- 'panic','misconduct','safety','no_show','complaint'
  severity      text not null,   -- 'low','medium','high','critical'
  description   text,
  action_taken  text,
  resolved_at   timestamptz,
  created_at    timestamptz not null default now()
);

create table ratings (
  booking_id      uuid primary key references bookings(id) on delete cascade,
  client_score    int check (client_score between 1 and 5),   -- client rates therapist
  client_comment  text,
  therapist_score int check (therapist_score between 1 and 5),-- therapist rates client
  therapist_note  text,                                       -- OPS ONLY, never shown
  tip_kes         int not null default 0,
  created_at      timestamptz not null default now()
);
```

TWO COLUMNS THAT MATTER COMMERCIALLY

`bookings.origin` and `clients.acquired_via` are how we calculate commission correctly and settle any dispute with the operating partner. They are not analytics nice-to-haves. Populate them on every write path, including ops-created bookings.

5. Row Level Security model

RLS is on for every table, without exception, from the first migration. A table with RLS disabled is a table exposed to anyone holding the anon key, which is shipped in the browser bundle.

5.1 Helper functions

```
create or replace function auth_role() returns user_role
language sql stable security definer set search_path = public as $$
  select role from profiles where id = auth.uid()
$$;

create or replace function is_staff() returns boolean
language sql stable as $$ select auth_role() in ('ops','admin') $$;
```

5.2 Policy matrix

Table	Client	Therapist	Ops / Admin	Anon
treatments, treatment_pricing	read	read	read/write	read
therapists (public view)	read active	read active	read/write	read active
therapist_credentials	—	read own	read/write	—
bookings	read/insert own	read assigned, update status only	full	—
clients	read/update own	read *limited view* for assigned booking	full	—
payments	read own	—	full	—
payouts	—	read own	full	—
incidents	—	insert own	full	—
ratings	read/write own (client fields)	write therapist fields	full	—

5.3 Representative policies

```
alter table bookings enable row level security;

create policy "clients read own bookings" on bookings for select
  using ( client_id = auth.uid() );

create policy "clients create own bookings" on bookings for insert
  with check ( client_id = auth.uid() and status = 'pending_payment' );

create policy "therapists read assigned bookings" on bookings for select
  using ( therapist_id = auth.uid() );

create policy "therapists update own booking status" on bookings for update
  using ( therapist_id = auth.uid() )
  with check ( therapist_id = auth.uid()
               and status in ('in_progress','completed') );

create policy "staff full access" on bookings for all
  using ( is_staff() ) with check ( is_staff() );
```

> **CLIENT PII EXPOSURE TO THERAPISTS**
>
> A therapist needs the client's first name, the address and the access notes — and nothing else. Do not grant `select` on `clients` to therapists. Expose a security-definer view (`therapist_booking_view`) that returns only those fields, only for bookings assigned to them, and only within a window around the appointment time. The client's phone number is never exposed; the therapist calls through the masked number (§9.3).

6. The availability & scheduling engine

One question, answered correctly: *given a treatment, a date, a delivery mode and a location, which therapist/time combinations can we actually honour?*

6.1 Inputs

Constraint	Source
Therapist declared working hours	`availability_windows` where `is_blocked = false`
Therapist time off	`availability_windows` where `is_blocked = true`
Existing bookings	`bookings.blocked_range`, statuses `confirmed` / `in_progress`
Travel time between consecutive jobs	`zone_travel_matrix`
Studio room capacity	`rooms` plus room-level conflict check
Skill requirement	`treatments.requires_skill` vs `therapist_skills`
Delivery mode eligibility	`therapists.serves_studio / serves_home / serves_hotel`
Client gender preference	Request parameter vs `therapists.gender`
Valid, unexpired credentials	`therapist_credentials` — see 6.2

6.2 Eligibility gate

Before any time arithmetic, reduce the therapist set. A therapist is eligible only if active, vetted, mode-capable, skill-matched, gender-matched, and holding current credentials of every required kind.

```
create or replace function eligible_therapists(
  p_treatment_id uuid, p_mode delivery_mode, p_gender text default null
) returns setof therapists
language sql stable as $$
  select t.* from therapists t
  where t.is_active
    and t.vetted_at is not null
    and case p_mode when 'studio' then t.serves_studio
                    when 'home'   then t.serves_home
                    else t.serves_hotel end
    and (p_gender is null or t.gender = p_gender)
    and not exists (                      -- no expired mandatory credential
      select 1 from therapist_credentials c
      where c.therapist_id = t.id
        and c.kind in ('tvet_cdacc','good_conduct','insurance')
        and (c.verified_at is null or c.expires_on < current_date)
    )
    and (
      (select requires_skill from treatments where id = p_treatment_id) is null
      or exists (select 1 from therapist_skills s
                 where s.therapist_id = t.id
                   and s.skill = (select requires_skill from treatments
                                  where id = p_treatment_id))
    );
$$;
```

6.3 Travel: use the zone table, not a maps API

Mombasa's geography is effectively linear: the island, then north through Nyali, Bamburi, Shanzu and on to Mtwapa, with a small number of crossing bottlenecks. A five-by-five lookup table captures this well enough and costs nothing.

```
-- Illustrative. Ops to tune these from real observed journeys after month one.
-- Values are one-way minutes, deliberately generous.
insert into zone_travel_matrix (from_zone, to_zone, minutes) values
 ('island','island',15),  ('island','nyali',25),  ('island','bamburi',40),
 ('island','shanzu',50),  ('island','mtwapa',60),
 ('nyali','nyali',15),    ('nyali','bamburi',20), ('nyali','shanzu',30),
 ('nyali','mtwapa',40),
 ('bamburi','bamburi',15),('bamburi','shanzu',15),('bamburi','mtwapa',25),
 ('shanzu','shanzu',15),  ('shanzu','mtwapa',15),
 ('mtwapa','mtwapa',15);
-- plus the symmetric inverse rows
```

RECOMMENDATION

Do not call the Google Distance Matrix API for availability queries. Availability is queried far more often than it is booked — every date the client browses fires a query — and per-request billing on a read-heavy path will be both slow and expensive. Revisit only if we expand beyond the coastal corridor.

6.4 Algorithm

```
function availableSlots(treatment, date, mode, zone, genderPref):

  therapists ← eligible_therapists(treatment, mode, genderPref)
  slotSize   ← 15 minutes
  results    ← []

  for each therapist T in therapists:
      working  ← union of T's non-blocked availability_windows on date
      busy     ← T's bookings on date, status in (confirmed, in_progress)

      for each candidate start S in working, stepping by slotSize:

          serviceEnd ← S + treatment.duration_min
          if serviceEnd not within working: continue

          -- Outbound leg: can T reach this job from the previous one?
          prev ← latest booking in busy ending at or before S
          if prev exists:
              travel ← travelMinutes(prev.zone, zone)
              if S < prev.service_end + prev.turnaround + travel: continue
          else:
              travel ← travelMinutes(T.home_zone, zone)
              if S < startOfWindow + travel: continue

          -- Return leg: can T reach the NEXT job after finishing this one?
          next ← earliest booking in busy starting at or after serviceEnd
          if next exists:
              back ← travelMinutes(zone, next.zone)
              if serviceEnd + treatment.turnaround + back > next.service_start:
                  continue

          -- Studio only: is a room free for the whole blocked range?
          if mode = 'studio':
              if no room free over [S, serviceEnd + turnaround]: continue

          results.append({ therapist: T, start: S, room: assignedRoom })

  return results grouped by start time
```

THE RETURN LEG IS THE STEP PEOPLE FORGET

Checking only that the therapist can *arrive* produces schedules where they cannot reach their next appointment. Both legs must be checked. Write a test for exactly this case.

6.5 Performance

With under ten therapists and a two-week booking horizon this runs comfortably in Postgres as a single stable SQL function called via RPC. Do not build a caching layer or a materialised availability table yet. Index for it:

```
create index bookings_therapist_range_idx on bookings
  using gist (therapist_id, blocked_range)
  where status in ('confirmed','in_progress');

create index availability_therapist_window_idx on availability_windows
  using gist (therapist_id, window);
```

Revisit only if p95 on the availability RPC exceeds 400 ms.

7. Booking transaction and double-booking prevention

7.1 Enforce it in the database

Two clients can hit "book" on the same slot in the same second. Application-level checking cannot prevent this reliably. Postgres exclusion constraints can, and they cost one line each.

```
create extension if not exists btree_gist;

-- A therapist cannot have two overlapping blocked ranges.
alter table bookings add constraint no_therapist_overlap
  exclude using gist (
    therapist_id with =,
    blocked_range with &&
  ) where (status in ('confirmed','in_progress'));

-- A studio room cannot be double-booked.
alter table bookings add constraint no_room_overlap
  exclude using gist (
    room_id with =,
    blocked_range with &&
  ) where (status in ('confirmed','in_progress') and room_id is not null);
```

> **WHY THIS MATTERS MORE THAN IT LOOKS**
>
> These two constraints make double booking *impossible*, regardless of any bug we later introduce in the availability query, the ops console, or a partner integration. Treat a constraint violation as an expected outcome and translate it into a friendly "that slot was just taken" message, not a 500.

7.2 The booking RPC

Booking is a multi-step operation that must be atomic: re-verify availability, compute price, compute the blocked range, insert. Do it in one `security definer` function so there is no window between check and write.

```
create or replace function create_booking(
  p_treatment_id uuid, p_therapist_id uuid, p_start timestamptz,
  p_mode delivery_mode, p_zone text, p_room_id uuid,
  p_address text, p_hotel text, p_room_no text
) returns bookings
language plpgsql security definer set search_path = public as $$
declare
  v_t treatments%rowtype;
  v_price int;
  v_travel int := 0;
  v_booking bookings%rowtype;
begin
  if auth.uid() is null then raise exception 'AUTH_REQUIRED'; end if;

  select * into v_t from treatments where id = p_treatment_id and is_active;
  if not found then raise exception 'TREATMENT_UNAVAILABLE'; end if;

  select price_kes into v_price from treatment_pricing
   where treatment_id = p_treatment_id and mode = p_mode and is_available;
  if v_price is null then raise exception 'PRICE_UNAVAILABLE'; end if;

  if p_mode <> 'studio' then
    select minutes into v_travel from zone_travel_matrix
     where from_zone = p_zone and to_zone = p_zone;   -- outbound buffer
  end if;

  insert into bookings (
    reference, client_id, therapist_id, treatment_id, mode, status, origin,
    service_range, blocked_range, room_id, zone_id,
    address_line, hotel_name, hotel_room_no, price_kes
  ) values (
    gen_booking_ref(), auth.uid(), p_therapist_id, p_treatment_id, p_mode,
    'pending_payment', 'app',
    tstzrange(p_start, p_start + make_interval(mins => v_t.duration_min), '[)'),
    tstzrange(p_start, p_start + make_interval(
        mins => v_t.duration_min + v_t.turnaround_min + v_travel), '[)'),
    p_room_id, p_zone, p_address, p_hotel, p_room_no, v_price
  ) returning * into v_booking;

  return v_booking;
exception
  when exclusion_violation then raise exception 'SLOT_TAKEN';
end; $$;
```

7.3 Holding the slot during payment

A booking is created as `pending_payment`, which the exclusion constraints deliberately ignore — otherwise abandoned checkouts would lock slots indefinitely. Instead:

- The client has **10 minutes** to complete payment.
- A pg_cron job every minute expires `pending_payment` bookings older than 10 minutes to `cancelled`.
- On successful payment the callback moves the booking to `confirmed`, at which point the exclusion constraints engage. If two people paid for the same slot in that window, the second transition raises `exclusion_violation` — refund automatically and notify ops. Rare, but it must be handled rather than crash.

7.4 Cancellation policy in code

```
-- Hours before service start → fee as % of price
--   > 12h   : 0%
--   2h – 12h: 50%
--   < 2h    : 100%
--   no-show : 100%
create or replace function cancellation_fee(p_booking_id uuid)
returns int language sql stable as $$
  select case
    when lower(service_range) - now() > interval '12 hours' then 0
    when lower(service_range) - now() > interval '2 hours'  then round(price_kes * 0.5)
    else price_kes
  end from bookings where id = p_booking_id;
$$;
```

Ops must be able to waive a fee, and every waiver is logged with the staff member's id. Build the audit trail now; it will be asked for.

8. Payments

8.1 Provider abstraction — build this first

We will change payment provider at least once, most likely from an aggregator to direct Safaricom Daraja once volume justifies the two-to-six week onboarding. Write the interface before the first integration.

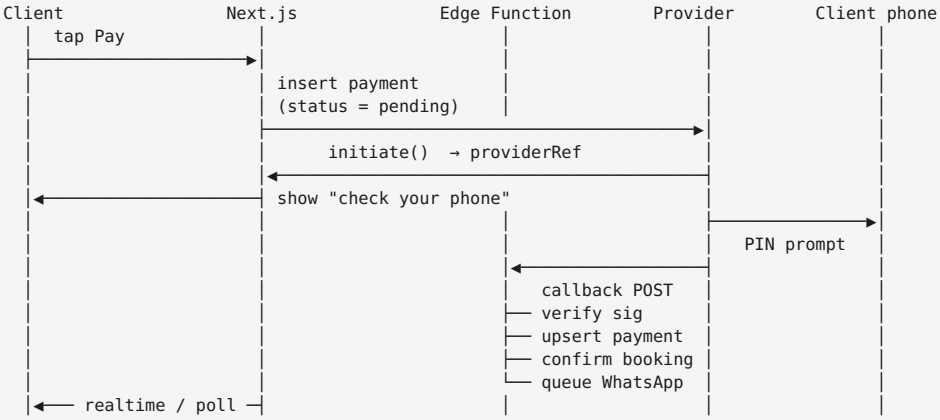
```
interface PaymentProvider {
  name: 'pesapal' | 'intasend' | 'dpo' | 'daraja';

  initiate(input: {
    bookingId: string; amountKes: number; phone?: string;
    method: 'mpesa' | 'card'; currency: 'KES' | 'USD' | 'EUR' | 'GBP';
  }): Promise<{ providerRef: string; redirectUrl?: string }>;

  verifySignature(headers: Headers, rawBody: string): boolean;
  parseCallback(rawBody: string): { providerRef: string; status: PaymentStatus;
                                    receipt?: string; amountKes?: number };
  queryStatus(providerRef: string): Promise<PaymentStatus>;   // reconciliation
  refund(paymentId: string, amountKes: number): Promise<void>;
  payout?(input: { phone: string; amountKes: number; ref: string }): Promise<void>;
}
```

Launch with Pesapal or IntaSend for M-Pesa (fastest to live, they handle token refresh, callbacks and reconciliation) and **DPO Pay** for cards and multi-currency, which matters for the hotel and tourist channel. Migrate M-Pesa to direct Daraja later for the lower per-transaction cost.

8.2 STK Push flow



8.3 The six failure paths — all must be tested

Scenario	Required behaviour
STK success	Booking → `confirmed`, WhatsApp confirmation sent, therapist notified
Client cancels the prompt	Payment → `failed`, booking stays `pending_payment`, client offered retry
STK timeout	Payment → `timed_out`. **Money may still have moved.** Never auto-cancel on timeout alone — the reconciliation sweep must confirm with the provider before we release the slot.
Card success	Same as STK success; store currency and FX rate applied
Card declined	Payment → `failed`, clear message, retry with a different method offered
Refund	Payment → `refunded`, booking → `cancelled`, audit row written

> **IDEMPOTENCY IS MANDATORY**
>
> Providers retry callbacks, sometimes several times, sometimes out of order, and occasionally a callback arrives before our `initiate()` call has returned. The `unique (provider, provider_ref)` constraint plus an upsert handles this. Never process a callback with a plain `insert`, and never assume a payment row already exists when the callback lands.

8.4 Reconciliation sweep

A pg_cron job every two minutes selects payments in `pending` or `timed_out` older than 120 seconds, calls `queryStatus()`, and settles them. This job is what protects us from silent money loss. Alert ops on any payment unresolved after 30 minutes.

8.5 Payouts

Weekly, Monday, covering the previous Monday to Sunday. For each therapist, sum completed bookings, apply `therapists.commission_rate`, add tips, produce a `payouts` row in `draft`.

- **Ops approves each batch manually before any money moves.** At this volume, automated disbursement is unnecessary risk. Automate later, once the numbers have been visibly correct for three months.
- Execution via provider B2C, or manually through the M-Pesa portal at launch — the console just needs to record the reference.
- Therapists see their own payout history in the therapist app, itemised by booking. This is a retention feature; make it clear and never late.

9. WhatsApp, notifications and number masking

> **PRIORITY NOTE**
>
> The WhatsApp layer is the highest-value item in the whole build. It is what reduces no-shows and drives rebooking. If schedule pressure forces a cut, cut the therapist earnings ledger before you cut reminders.

9.1 Message catalogue

Trigger	Channel	Content
Booking confirmed	WhatsApp	Reference, treatment, date/time, therapist name, location, price paid, cancellation policy
T−24 hours	WhatsApp	Reminder, with one-tap reschedule and cancel links
T−2 hours	WhatsApp	Final reminder; for home visits, "please confirm the address"
Therapist en route	WhatsApp	Triggered by therapist app; therapist name, photo, ETA
Session complete	WhatsApp	Thank you, rating link, optional tip
Rebooking nudge	WhatsApp	21 days after last completed session
New job offer	WhatsApp + push	To therapist: treatment, time, zone, fee
Any of the above	SMS fallback	Via Africa's Talking if WhatsApp delivery fails or the number is not on WhatsApp

9.2 Implementation

- WhatsApp Business Cloud API. Pre-approve message templates early — approval takes days and is a common launch blocker.
- Outbound messages go through a `notifications` queue table, dispatched by pg_cron every minute. Never send inline in a request handler; a WhatsApp outage must not fail a booking.
- Store `sent_at`, `delivered_at`, `failed_at` and the provider message id. Retry twice, then fall back to SMS.
- Inbound webhook lands on an Edge Function. At launch, route all inbound messages to the ops console rather than attempting a bot. Conversational booking is phase two at the earliest.

9.3 Number masking

Required before mobile (home and hotel) service goes live. Client and therapist must be able to call each other about access, parking or delays without exchanging real numbers.

- Africa's Talking voice proxy: allocate a pool of virtual numbers; bind a pair to a booking from T−2h to T+2h; route calls through and log them.
- After the window closes, the binding is released and the number returns to the pool.
- Call metadata (who, when, duration) is logged. Content is not recorded.

This is a genuine commercial control as well as a privacy one: it is the main technical brake on clients and therapists taking the relationship off-platform.

10. Edge Functions and background jobs

10.1 Edge Functions

Function	Auth	Purpose
`payment-callback`	Signature	Receives provider callbacks; verifies, upserts payment, confirms booking, queues notifications
`whatsapp-webhook`	Meta verify token	Inbound messages and delivery receipts
`initiate-payment`	User JWT	Creates the payment row and calls the provider. Holds provider secrets.
`panic-alert`	User JWT	Writes a critical incident, SMSes the on-call responder, shares last known GPS
`voice-proxy`	Provider signature	Masked-call routing callback

> **PUBLIC ENDPOINTS**
>
> Callback functions are internet-facing and unauthenticated by design. Every one must verify a provider signature or shared secret before touching the database, and must be safe to call twice with the same payload. Treat these as the highest-risk surface in the system.

10.2 Scheduled jobs (pg_cron)

Job	Schedule	Action
`expire_pending`	every minute	Cancel `pending_payment` bookings older than 10 minutes
`dispatch_notifications`	every minute	Send queued WhatsApp/SMS, handle retries
`queue_reminders`	every 15 min	Enqueue T−24h and T−2h reminders
`reconcile_payments`	every 2 min	Query provider for unresolved payments
`session_watchdog`	every 5 min	Flag sessions past expected end with no check-out → ops alert
`credential_expiry`	daily 06:00	Warn ops 30 days before any therapist credential expires
`build_payouts`	Mondays 02:00	Generate draft payout rows for the prior week

`session_watchdog` is a safety control, not a housekeeping job. A therapist who has not checked out fifteen minutes past the expected end of a home visit must produce a phone call from ops within five minutes. Build the alert path and test it.

11. Security, privacy and Kenyan obligations

11.1 What we hold

Names, phone numbers, home addresses, ID or passport numbers, payment references, appointment histories and therapist health notes. Under Kenyan law that is a serious data set, and appointment history combined with home address is materially sensitive.

11.2 Compliance requirements

Obligation	What engineering must deliver
ODPC registration	Business action, but engineering supplies the processing description. Required where turnover exceeds KES 5M, staff exceed 10, sensitive data is processed, or data of 10,000+ subjects is handled in a year.
Data Protection Impact Assessment	Must be completed before processing begins. Engineering documents data flows, retention and third-party processors.
Breach notification	72 hours to the ODPC. We need audit logging good enough to establish scope quickly, and a documented escalation path.
Consent	Freely given, specific, informed, unambiguous. Separate, unticked checkboxes for terms, marketing and health-note storage. No pre-ticked boxes, no bundling.
Data subject rights	Export and deletion endpoints in the ops console. Deletion anonymises rather than drops booking rows, so financial records survive.

11.3 Engineering controls

- **RLS on every table**, verified by an automated test that fails CI if any public table has RLS disabled.
- **Service role key server-side only.** Add a CI grep that fails the build if it appears in any client bundle.
- **Encrypt ID document numbers** at the column level using `pgcrypto`, with the key in Supabase Vault. Never log them.
- **Private storage bucket** for credential scans, accessed only through short-lived signed URLs generated server-side for ops.
- **Audit log** table capturing every staff read of client PII, every fee waiver, every ban and every payout approval, with actor and timestamp.
- **Retention:** GPS check-in/out points purged after 90 days; masked-call metadata after 90 days; booking and payment records retained seven years for tax.
- **Secrets** in Vercel and Supabase environment configuration only. Never in the repository, never in the client bundle.
- **Rate limiting** on OTP requests and the availability RPC.

> **THERAPIST SAFETY DATA**
>
> Panic-button events, GPS traces and incident records exist to protect people. Access is ops-and-admin only, every access is logged, and this data is never used for performance management. Make that explicit in the therapist-facing privacy notice — if therapists suspect the panic button is surveillance, they will not use it, and the safety architecture fails.

12. Environments, CI/CD and testing

12.1 Environments

Environment	Supabase	Web	Payments
Local	Supabase CLI, Docker	`next dev`	Provider sandbox
Staging	`wellness-staging`	Vercel preview	Provider sandbox
Production	`wellness-prod`	Vercel production	Live credentials

12.2 Practices

- All schema changes as migrations in the repository via the Supabase CLI. **No changes through the dashboard on production, ever.** A dashboard change that is not in a migration will be silently lost on the next deploy and will be very hard to diagnose.
- Trunk-based development, short-lived branches, Vercel preview per pull request.
- Seed script producing realistic data: five therapists, twenty treatments, two rooms, a month of bookings. Everyone develops against the same fixtures.
- Error tracking (Sentry) from week one, not after the first incident.

12.3 Test coverage that actually matters

Do not chase a coverage percentage. Cover these specific cases, and treat any of them failing as a release blocker:

1. Two concurrent bookings for the same therapist and slot → exactly one succeeds, the other receives `SLOT_TAKEN`.
2. Two concurrent studio bookings for the last free room → same.
3. Travel outbound: a Nyali booking ending 14:00 must not offer a Shanzu slot at 14:15.
4. **Travel return leg:** a slot must not be offered if taking it would make the therapist late for an existing later booking.
5. Therapist with an expired good-conduct certificate disappears from availability.
6. All six payment paths in §8.3.
7. Duplicate payment callback processed twice → one payment row, one confirmation message.
8. Callback arriving before `initiate()` returns → handled without error.
9. Cancellation fees at 13h, 6h and 1h before start → 0%, 50%, 100%.
10. RLS: a therapist cannot read a booking assigned to another therapist; a client cannot read another client's booking. Test with real JWTs, not the service role.
11. Panic button writes an incident and dispatches an SMS within 10 seconds.

13. 12-week delivery plan

Weeks	Focus	Deliverables
1–2	Foundations	Supabase projects, full schema with RLS and exclusion constraints, auth with phone OTP, seed data, CI, Vercel deploys, payment provider interface (no implementation), Sentry
3–5	Booking core	Treatment catalogue with three-mode pricing, `eligible_therapists`, availability RPC with zone travel and both legs, booking RPC, client booking flow end to end, therapist profiles with credentials displayed, cancellation policy
6–7	Payments	M-Pesa via aggregator, card via DPO, callback Edge Function with signature verification and idempotency, reconciliation sweep, refunds, all six failure paths tested
8–9	Therapist app	Availability management, job offers and acceptance, check-in/out with GPS, panic button and alert path, earnings ledger, session watchdog
10–11	WhatsApp & ops	Notification queue and dispatcher, all seven message templates approved and live, SMS fallback, number masking, ops console (live board, manual booking, exception handling, incident log, draft payout runs)
12	Hardening & launch	Partner booking links with commission tracking, staff training, load sanity check, DPIA documented, existing clients migrated, soft launch

> **DEPENDENCIES TO START IN WEEK ONE**
>
> Three items have external lead times that will otherwise block launch: WhatsApp template approval (days to weeks), payment provider onboarding and live credentials (2–6 weeks for direct Daraja, faster via an aggregator), and Africa's Talking virtual number allocation. Begin all three in week one, before the code that needs them exists.

14. Definition of done

A feature ships when all of the following are true. No exceptions for launch pressure.

- Schema changes are migrations in the repository, applied to staging first.
- RLS policies exist and are covered by a test using a real user JWT.
- Errors surface as user-readable messages, never as a raw Postgres error string.
- Every write path populates `bookings.origin` and, for new clients, `clients.acquired_via`.
- Anything touching money is idempotent and has a reconciliation path.
- Anything touching client PII is logged in the audit table.
- The flow has been walked through on a real mid-range Android phone on a mobile connection — not only on a laptop. Most of our clients and all of our therapists will be on exactly that.
- Ops can perform the equivalent action manually in the console when the automated path fails.

Open questions for the team

1. Confirm the Supabase region after latency testing from Mombasa: Frankfurt or Mumbai.
2. Confirm the payment aggregator: Pesapal or IntaSend for M-Pesa. Whichever onboards us faster.
3. Decide whether therapists accept or are assigned jobs. The spec assumes *client picks from available therapists*, which supports the trust positioning but means we must handle a therapist declining after selection.
4. Confirm the studio room count so the room-conflict logic is correct from day one.
5. Agree the initial zone travel matrix values with ops, and schedule a review after one month of real journeys.